

«Мы можем видеть лишь небольшое расстояние впереди, но уже видим, как много там нужно сделать.»

— Алан Тьюринг

Куратор спрашивает, что машина может вычислить в принципе, что не может никогда и каким задачам не хватает времени. Инструменты — машины Тьюринга, диагональное рассуждение, сведение задач и классы сложности.

Задача 1: Машина Тьюринга — инкремент

База

Машина работает так: головка ходит по ленте, читает и пишет символы, а её поведение задано состояниями и переходами. Куратор приспособливает её под скучную, но нужную работу: прибавлять единицу к двоичной записи числа, лежащей на ленте (младший разряд слева, число ограничено пробелами).

1. Напишите состояния и переходы машины (головка стартует на младшем — левом — разряде): как она прибавляет единицу и останавливается.
2. Проследите её работу на записи «1101» (число 11), выписывая конфигурации на каждом шаге (что на ленте, где головка, какое состояние).
3. Проследите на «111» (число 7): что происходит, когда перенос проходит через все разряды?

Задача 2: Конфигурации и язык $a^n b^n$

База

Машине поручено распознавать слова вида $a^n b^n$: сначала n букв a , затем n букв b . Такое слово — пароль доступа к хранилищу, и машина должна останавливаться, приняв правильное слово, и отвергнуть неправильное.

1. Постройте машину: помечайте первую непомеченную a , идите направо, помечайте первую непомеченную b , возвращайтесь и повторяйте. В конце проверьте, что непомеченных не осталось, а порядок a^*b^* соблюден.
2. Выпишите последовательность конфигураций построенной машины на входе «aaabbb» до остановки.
3. Сколько шагов она сделает на слове $a^n b^n$? Выразите через n .
4. Что произойдёт на «aabbb» (два a , три b): распознает ли машина слово? Проследите до остановки и объясните, на каком шаге она отвергает слово.

Задача 3: Распознавание и разрешение

Челлендж

Куратору нужен полный список слов, принимаемых машиной. Список можно строить, прогоняя слова через машину, но машина на некоторых словах может работать вечно, не давая ответа.

1. Докажите: если язык машины и его **дополнение** можно порождать перечислением, то язык машины **разрешается** — для любого слова машина (построенная по двум перечислителям) всегда остановится с верным «да/нет».
2. Дайте конкретный пример языка, который порождается перечислением (распознаётся), но не разрешается.
3. Приведите пример, когда перечисление дополнения нельзя получить, прогоняя слова через саму машину, но язык при этом разрешается.

Задача 4: Проблема остановки

Челлендж

Куратор хочет программу, которая по описанию *любой* машины и *любому* входу отвечает, остановится ли она. Если бы такая программа была, она решала бы все вопросы о поведении машин сразу, и именно это, как показал Тьюринг, невозможно.

1. Постройте машину D , которая на собственном коде приводит этот вопрос к противоречию, и покажите, что такой программы не существует.
2. Покажите в точности, где рассуждение опирается на допущение «программа всегда отвечает», и что ломается, если разрешить ей не отвечать.
3. Объясните, почему то же диагональное рассуждение даёт несчётность $\mathbb{R}$ — в чём аналогия.

Задача 5: Машины Тьюринга счётны

База

Каждая машина Тьюринга — конечный набор переходов, а любую конечную запись можно переписать строкой символов.

1. Занумеруйте все машины Тьюринга натуральными числами и обоснуйте, что такая нумерация возможна.
2. Сравните число машин с числом всех языков над одним алфавитом и сделайте вывод о существовании языков, которые никакая машина не распознаёт.
3. Что этот подсчёт говорит о «пределе» вычислимого?

Задача 6: Сводимость

База

Сведение — это способ переписать одну задачу в другую так, что ответ на вторую даёт ответ на первую. Именно так доказывают неразрешимость: если бы решалась вторая задача, решалась бы и неразрешимая первая.

1. Сведите проблему остановки к вопросу «пуст ли язык машины» (не принимает ни одного слова) и объясните, почему сведение — доказательство неразрешимости.
2. Сведите вопрос «принимает ли машина слово w » к «останавливается ли машина на входе x ».
3. Объясните, зачем вообще нужны сведения, если проблема остановки уже неразрешима.

Задача 7: Теорема Райса

Челлендж

Куратор хочет по одному лишь описанию машины понять её поведение. Останавливается ли машина ровно на одном входе? Печатает ли она хоть один символ? Теорема Райса говорит, что любое нетривиальное свойство вычислимой функции неразрешимо.

1. Докажите, что невозможно по описанию машины решить, останавливается ли она ровно на одном входе (сведите этот вопрос к проблеме остановки).
2. Докажите, что вопрос «печатает ли машина хоть один символ» неразрешим, сведя его к проблеме остановки (это свойство запуска, а не вычислимой функции, поэтому теорема Райса к нему напрямую не применима).
3. Приведите пример *разрешимого* свойства машин — и объясните, почему оно разрешимо, а предыдущие — нет.

Задача 8: Сертификаты и верификация

База

Куратору нужно понять, есть ли в сети хранилищ вершинное покрытие размера не более k — набор узлов, задевающий каждое соединение. Проверять «да» легко, если кто-то предъявил готовое покрытие: трудность в том, чтобы его *найти*.

1. Опишите *свидетель* (сертификат) и *проверяющую* программу, которая за полиномиальное время проверяет, что предъявленное покрытие действительно размера $\leq k$ и задевает все рёбра.
2. Покажите, что и задача «есть ли покрытие размера $\leq k$ », и задача «есть ли покрытие *ровно* k » имеют полиномиальный сертификат. Различие здесь лишь в простой проверке размера. Объясните, почему сам факт наличия сертификата ничего не говорит о сложности *поиска*.
3. Объясните, почему класс задач с полиномиальным сертификатом содержит класс задач, решаемых за полиномиальное время.

Задача 9: Недетерминированные машины

База

Если разрешить машине *угадывать* — например, в каждой развилке идти по всем вариантам сразу, — она может «найти» решение, не перебирая его.

1. Постройте недетерминированную машину, которая за полиномиальное время угадывает вершинное покрытие размера k и проверяет его.
2. Оцените, во что обходится детерминированное моделирование такого угадывания: выразите время через k и число вершин.
3. Объясните, почему недетерминизм даёт выигрыш во времени, но не в выразительной мощности.

Задача 10: Сведение SAT к 3-SAT

Челлендж

Любую логическую формулу можно записать в конъюнктивной нормальной форме (КНФ), но для практики удобнее, когда в каждом дизъюнкте ровно три литерала.

1. Покажите, как любую КНФ-формулу преобразовать в 3-КНФ с сохранением выполнимости: длинный дизъюнкт разбейте вспомогательной переменной, а короткие — «размножьте».
2. Объясните, почему из NP-полноты SAT и этого сведения следует NP-полнота «3-SAT».
3. Почему достаточно доказать NP-полноту одной задачи, а остальные получать сведениями?

Задача 11: Сведение 3-SAT к вершинному покрытию

Челлендж

Куратор сводит логическую задачу к задаче о графе: по формуле φ с n переменными и m дизъюнктами он строит сеть хранилищ и число k так, что покрытие размера k существует тогда и только тогда, когда φ выполнима.

1. Опишите конструкцию. Каждой переменной x_i соответствует ребро из двух «значений» x_i и $\neg x_i$. Каждому дизъюнкту соответствует треугольник из трёх литеральных вершин, каждая соединена со своим «значением». Укажите, что $k = n + 2m$.
2. Покажите эквивалентность подробно, разбирая обе стороны.

3. Подсчитайте число вершин и рёбер полученного графа через n и m и подтвердите полиномиальность сведения.

Задача 12: P vs NP — что на кону

База

Класс задач, решаемых быстро, и класс задач, у которых ответ легко *проверить*, — совпадают ли они? Если да, то всё, что можно быстро проверить, можно и быстро найти.

1. Перечислите три практических последствия равенства $P = NP$: что станет с шифрованием, оптимизацией, автоматическим доказательством теорем.
2. Объясните, почему NP-полнота — плохая новость, но не приговор: приведите пример, когда NP-полную задачу всё равно решают на практике (и чем).
3. Что именно мешает просто «сдаться» и считать, что $P \neq NP$, и почему это открытый вопрос?

Задача 13: Задачи с коротким подтверждением и коротким опровержением

Челлендж

Есть задачи, где и «да», и «нет» можно убедительно обосновать коротким свидетельством. Проверка простоты числа — одна из них: и простоту, и составность можно доказать, предъявив небольшой объект.

1. Для проверки простоты числа приведите и короткое доказательство простоты, и короткое доказательство составности.
2. Объясните, почему у задач, где и «да», и «нет» имеют полиномиальный сертификат, не может быть NP-полноты (если только $NP \neq coNP$).
3. Приведите пример ещё одной задачи с «двусторонним сертификатом» (берите задачу не из класса P : иначе сертификат есть, но пример не показателен для $NP \cap coNP$).

Задача 14: Цена вычислений

Челлендж

Куратору нужно знать, во что на самом деле обходится алгоритм.

1. Оцените, сколько примерно операций нужно для перебора 2^n при $n = 100$ и для алгоритма за n^3 при $n = 10^6$, и во сколько раз они различаются.
2. Приведите пример задачи, для которой известен только экспоненциальный алгоритм, и объясните, почему NP-трудная задача не обязана быть «нерешаемой».
3. Объясните, почему многоленточная машина не быстрее одноленточной по выразительной мощности, но может быть быстрее по времени, и оцените, во сколько раз.

Необязательные бонусные задачи

Бонусные задачи не входят в обязательный минимум. Они заметно сложнее основных и требуют либо программирования, либо самостоятельного обобщения материала. Оценивается в них прежде всего объяснение модели, её корректности и границ применимости.

Задача А: Минимальные и универсальные машины Тьюринга

Бонус

Куратору мало «гонять» машины — он хочет самые компактные машины и одну, которая исполняет все остальные.

1. *Модель — часть задачи.* Эмулятор (лента, состояния, функция перехода) — отправная точка: возьмите готовый симулятор или реализуйте заново шаг и запуск до остановки. При превышении предела шагов останавливайтесь с пометкой «не завершилась за лимит».
2. Минимизация: для построенных выше машин — инкремента и распознавателя $a^n b^n$ — попробуйте сократить число состояний (слияние неразличимых состояний или перебор всех машин с меньшим числом состояний, по возрастанию размера). Сведите в таблицу размеры машин до и после минимизации.
3. Универсальность: закодируйте машину строкой (состояния и переходы) и постройте машину, которая по коду и входу воспроизводит работу закодированной машины. Сверьте прямой и универсальный прогон: вердикты и итоговые ленты должны совпадать.
4. Оформите отчёт: модель, таблица минимизации, кодирование и универсальная машина, ограничения.

Задача В: Полиномиальная иерархия и PSPACE

Бонус

Классы P и NP — только начало лестницы сложности: дальше идут $\Sigma_1 = NP$, $\Pi_1 = coNP$, $\Sigma_2 = NP^{NP}$, Π_2 , и так далее, а выше всех — PSPACE.

1. Определите классы Σ_k и Π_k через NP с оракулом (или через чередование кванторов) и выпишите первые три уровня.
2. Докажите, что $\Sigma_1 = NP$, $\Pi_1 = coNP$, и включения $\Sigma_k \subseteq \Pi_{k+1}$, $\Pi_k \subseteq \Sigma_{k+1}$.
3. Приведите пример задачи из Σ_2 (например, «существует ли раскраска графа, при которой нет разноцветного треугольника», то есть схема «существует ..., для всех ...»).
4. Объясните, почему PSPACE содержит всю иерархию, и приведите пример PSPACE-полной задачи (например, «выигрывает ли первый игрок в игру с конечным полем»).
5. Оформите отчёт: определения через кванторы, цепочка включений, примеры с позиций «существует/для всех».

Задача С: Почему диагональ не доказывает $P \neq NP$

Бонус

Куратор объясняет, почему простой диагональный трюк отделяет перечислимое от неразрешимого, но не отделяет P от NP .

1. Объясните, почему диагональный аргумент (как в проблеме остановки) не отделяет P от NP : что именно «ломается», если применять его внутри полиномиального времени?

2. Расскажите о время-иерархии: существуют функции, вычислимые за большее время, но не за меньшее. Как это согласуется с диагональю и почему из этого не следует $P \neq NP$?
3. Приведите пример «самореференции», которая даёт нижнюю оценку, и объясните, где она не работает для NP.